

## 《智慧邮件系统项目计划书》

作 者： 李明轩

完成日期： 2026/6/6

签 收 人：

签收日期：

修改情况记录：

| 版本号 | 修改批准人 | 修改人 | 签收人 |
|-----|-------|-----|-----|
|     |       |     |     |
|     |       |     |     |
|     |       |     |     |

## 目 录

- 1. 项目背景、范围及目标 ..... 错误!未定义书签。
- 2. 项目可行性分析 ..... 错误!未定义书签。
- 3. 项目实施计划 ..... 错误!未定义书签。
- 4. 项目任务分解结构（WBS） ..... 错误!未定义书签。

## 1. 项目背景、范围及目标

### 1.1 项目背景与意义

SmartMail 是基于实训课程要求开发的 Web 版智能邮件系统项目。在数字化时代，邮件仍是重要的沟通工具，本项目旨在通过设计和实现一个完整的邮件系统，让学生掌握：

- 全栈开发能力：前端、后端、AI 服务的协同开发
- 系统架构设计：模块化设计、分层架构、接口规范
- 数据库设计：关系模型建模、表设计、数据分离存储
- 插件化设计：如何优雅地集成 AI 能力而不破坏基础系统

### 1.2 系统基本功能

#### 1.2.1 核心邮件功能

- 用户认证：邮箱注册、登录、登出、JWT 鉴权
- 邮件收发：发送邮件、接收邮件、拉取邮件列表
- 邮箱管理：收件箱、已发送、草稿、垃圾箱
- 邮件操作：标记已读/未读、星标、删除、搜索、筛选
- 附件管理：上传附件、下载附件、附件元信息管理
- 标签管理：创建标签、给邮件打标签、标签筛选

#### 1.2.2 智能 AI 功能（加分项）

- 邮件摘要：使用 LLM 提取邮件要点
- 回复草稿：AI 生成可编辑的邮件回复
- 优先级预测：自动判断邮件优先级
- 标签建议：AI 推荐适用的标签
- 垃圾邮件识别：规则或 AI 识别垃圾邮件

### 1.3 要解决的问题

| 问题       | 解决方案                                         |
|----------|----------------------------------------------|
| 邮件跨用户可见性 | 邮件主数据和用户邮箱视图分离 (mail_message + mailbox_item) |
| 附件存储与性能  | 文件存磁盘，数据库保存元信息                               |
| AI 依赖性   | 可插拔架构，Agent 关闭时系统仍可用                         |
| 跨邮件服务器投递 | 首版仅支持同服务器用户间通信                               |
| 协议复杂性    | 采用 HTTP REST 而非 SMTP/POP3                    |

### 1.4 项目目标

首版交付目标（必做）：

1. 邮件收发闭环：完整的注册、发送、接收、查看流程
2. 邮箱管理：收件箱、已发送、草稿、垃圾等基础功能
3. 数据分离：邮件正文入库，附件存磁盘
4. 搜索与筛选：关键词搜索、文件夹筛选、已读状态筛选
5. 基础标签：用户自定义标签和邮件标签关系

展示型 AI 目标（展示项）：

1. 邮件摘要生成
2. 回复草稿生成
3. 优先级和标签建议

加分项目标：

1. 新邮件通知（SSE/WebSocket）

- 2. 自然语言搜索
  - 3. 垃圾邮件识别
  - 4. 用户行为统计
- 

2. 项目可行性分析

2.1 经济可行性

| 项目   | 评估              |
|------|-----------------|
| 开发成本 | 低。采用开源技术栈，无许可费用 |
| 硬件成本 | 低。单机或云服务器可满足    |
| 维护成本 | 低。基础邮件功能相对成熟    |
| 投资回报 | 教学价值高，展示效果好     |

结论：经济可行性强

2.2 技术可行性

2.2.1 技术栈成熟度

| 技术                  | 成熟度 | 评估               |
|---------------------|-----|------------------|
| Vue 3 + Vite        | 5   | 前端主流框架           |
| Spring Boot 3       | 5   | 企业级稳定框架          |
| MyBatis-Plus        | 5   | 国内广泛应用           |
| MySQL 8             | 5   | 业界标准数据库          |
| FastAPI             | 4   | 现代 Python Web 框架 |
| JWT                 | 5   | 业界标准认证方案         |
| OpenAI/DeepSeek SDK | 4   | 成熟的 LLM 调用库      |

结论：技术栈经过验证，无重大风险

2.2.2 核心难点与应对方案

| 难点           | 风险等级 | 应对方案                |
|--------------|------|---------------------|
| 邮件主数据与用户视图分离 | 中    | 提前完成数据库设计评审，建立 ER 图 |
| 多人并发开发代码冲突   | 中    | 规范接口契约，分模块开发，定期集成   |
| 前后端联调延迟      | 中    | 编写接口文档，提前 Mock 数据   |

|            |   |                            |
|------------|---|----------------------------|
| AI 服务集成复杂性 | 中 | 设计清晰的 Internal API，逐步集成    |
| 附件存储和下载    | 低 | 参考开源方案，采用标准路径管理            |
| 跨浏览器兼容性    | 低 | Element Plus 已处理，优先 Chrome |

结论：技术风险可控，无不可克服的障碍

## 2.3 运行可行性

### 2.3.1 团队能力评估

- 小组成员具备 Java/Vue/Python 基础
- 团队有全栈开发经验
- 项目周期充裕（5-6 周）
- 已有详细的架构和设计文档支撑

### 2.3.2 资源需求

| 资源     | 需求                           | 可得性 |
|--------|------------------------------|-----|
| 开发环境   | JDK 17、Node 18+、Python 3.10+ | 可用  |
| 开发工具   | IDE、Git、MySQL                | 可用  |
| 服务器资源  | 单机或云主机（1 核 2GB 足够）           | 可用  |
| API 额度 | OpenAI/DeepSeek Key（可选）      | 可用  |
| 文档工具   | Markdown、Draw.io             | 可用  |

结论：资源充足，运行可行性强

## 2.4 可行性总结

|          |       |              |
|----------|-------|--------------|
| 1. 维度    | 2. 评分 | 3. 结论        |
| 4. 经济    | 5. 5  | 6. 完全可行      |
| 7. 技术    | 8. 4  | 9. 可行，风险可控   |
| 10. 运行   | 11. 5 | 12. 完全可行     |
| 13. 综合评价 | 14. 5 | 15. 项目强烈推荐启动 |

### 3. 项目实施计划

#### 3.1 开发周期与阶段划分

项目总周期 5-6 周，分为 5 个阶段：

| 阶段   | 周数    | 主要工作                  | 交付物                            |
|------|-------|-----------------------|--------------------------------|
| 第一阶段 | W1    | 需求分析、架构设计、数据库设计、项目初始化 | 需求文档、架构图、ER 图、接口文档             |
| 第二阶段 | W2    | 后端基础、用户认证、邮件收发基础      | Auth 模块、Mail 模块                |
| 第三阶段 | W2-W3 | 邮箱管理、附件、前端初始化         | Mailbox 模块、Attachment 模块、UI 框架 |
| 第四阶段 | W3-W4 | 搜索标签、前端联调、AI 接入       | Search、Label 模块、Agent 服务、前端页面  |
| 第五阶段 | W4-W5 | 测试、优化、文档补充、答辩准备       | 测试报告、演示数据、答辩 PPT               |

#### 3.2 详细开发时间表（甘特图）

...

| 阶段   | 任务项   | W1  | W2 | W3 | W4 | W5 |
|------|-------|-----|----|----|----|----|
| 阶段 1 | 需求分析  | 1   | 0  | 0  | 0  | 0  |
|      | 架构设计  | 0.6 | 0  | 0  | 0  | 0  |
|      | 数据库设计 | 0.4 | 0  | 0  | 0  | 0  |
|      | 项目初始化 | 1   | 1  | 0  | 0  | 0  |

|      |            |   |     |     |     |     |
|------|------------|---|-----|-----|-----|-----|
| 阶段 2 | 后端框架搭建     | 0 | 1   | 0   | 0   | 0   |
|      | 用户认证模块     | 0 | 0.5 | 0.5 | 0   | 0   |
|      | 邮件收发核心功能   | 0 | 1   | 1   | 0   | 0   |
| 阶段 3 | 邮箱管理模块     | 0 | 0   | 1   | 0   | 0   |
|      | 附件管理模块     | 0 | 0   | 0.5 | 0.5 | 0   |
|      | 前端框架初始化    | 0 | 0   | 1   | 0   | 0   |
|      | 主布局与基础组件   | 0 | 0   | 0.5 | 0.5 | 0   |
| 阶段 4 | 标签与搜索功能    | 0 | 0   | 0   | 1   | 0   |
|      | AI 能力集成    | 0 | 0   | 0   | 0.5 | 0.5 |
|      | 前端核心页面开发   | 0 | 0   | 0   | 1   | 1   |
|      | Agent 功能开发 | 0 | 0   | 0   | 0.5 | 0.5 |
| 阶段 5 | 全流程测试      | 0 | 0   | 0   | 0   | 1   |
|      | 优化与问题修复    | 0 | 0   | 0   | 0   | 0.5 |
|      | 项目文档补充     | 0 | 0   | 0   | 0   | 1   |
|      | 项目答辩与演示    | 0 | 0   | 0   | 0   | 0.5 |

### 3.3 关键里程碑与节点

| 日期    | 里程碑       | 验收标准              |
|-------|-----------|-------------------|
| W1 结束 | 架构设计完成    | 有接口文档、ER 图、API 规范 |
| W2 结束 | 邮件收发基础闭环  | 能发送、能查看、能读取       |
| W3 结束 | 前端基础框架完成  | 能集成后端接口，邮件列表可显示   |
| W4 结束 | AI 模块可选启动 | 摘要、草稿、优先级功能可用     |
| W5 结束 | 项目完成可演示   | 所有必做功能完成，可答辩      |

### 3.4 并行开发策略

为了缩短周期，采用 并行开发 模式：

- W1：架构师设计文档，同时后端开始 Spring Boot 初始化

- W2: 后端开发认证和邮件模块，前端等待接口文档
- W3: 前端并行开发 UI，后端继续模块开发
- W4: 前端联调后端接口，AI 团队单独推进 Agent
- W5: 集成测试、性能优化、答辩准备

---

## 4. 项目任务分解结构（WBS）

### 4.1 WBS 分解图

```

SmartMail 项目

#### 1. 需求与设计

- 1.1 需求分析文档
- 1.2 系统架构设计
- 1.3 数据库 ER 图与设计
- 1.4 接口设计文档
- 1.5 项目规划与风险评估

#### 2. 后端开发（Spring Boot）

- 2.1 项目初始化与基础设施
  - 2.1.1 Spring Boot 工程搭建
  - 2.1.2 MySQL 数据库初始化
  - 2.1.3 通用响应与异常处理
  - 2.1.4 日志与配置管理
- 2.2 认证与安全（auth 模块）
  - 2.2.1 用户注册接口
  - 2.2.2 用户登录接口
  - 2.2.3 JWT Token 生成与验证
  - 2.2.4 登出与 Token 刷新
  - 2.2.5 Spring Security 集成
- 2.3 用户模块（user 模块）
  - 2.3.1 用户资料查询
  - 2.3.2 用户信息更新
  - 2.3.3 当前用户接口
- 2.4 邮件核心（mail 模块）
  - 2.4.1 发送邮件接口
  - 2.4.2 邮件详情查询
  - 2.4.3 收件人关系管理
  - 2.4.4 邮件正文处理
- 2.5 邮箱管理（mailbox 模块）
  - 2.5.1 邮件列表查询（分页、筛选）
  - 2.5.2 标记已读/未读
  - 2.5.3 星标邮件
  - 2.5.4 删除到垃圾箱
  - 2.5.5 恢复邮件

- 2.6 附件模块 (attachment 模块)
  - 2.6.1 附件上传接口
  - 2.6.2 附件下载接口
  - 2.6.3 附件元信息管理
  - 2.6.4 磁盘存储管理
- 2.7 标签模块 (label 模块)
  - 2.7.1 创建标签
  - 2.7.2 编辑标签
  - 2.7.3 删除标签
  - 2.7.4 给邮件打标签
  - 2.7.5 标签筛选
- 2.8 搜索模块 (search 模块)
  - 2.8.1 主题/正文关键词搜索
  - 2.8.2 发件人搜索
  - 2.8.3 组合条件搜索
  - 2.8.4 日期范围筛选
  - 2.8.5 已读状态筛选
- 2.9 AI 门面 (ai 模块)
  - 2.9.1 AI 任务管理
  - 2.9.2 邮件摘要接口
  - 2.9.3 回复草稿接口
  - 2.9.4 优先级与标签建议接口
  - 2.9.5 AI 结果查询
  - 2.9.6 AI 开关配置
- 2.10 内部工具 API (internal 模块)
  - 2.10.1 邮件查询工具
  - 2.10.2 标签应用工具
  - 2.10.3 优先级设置工具
  - 2.10.4 AI 结果保存工具
  - 2.10.5 内部 Token 认证
- 3. AI Agent 开发 (Python)
  - 3.1 Agent 项目初始化
    - 3.1.1 FastAPI 框架搭建
    - 3.1.2 Pydantic 数据模型定义
    - 3.1.3 配置管理
  - 3.2 LLM 集成
    - 3.2.1 OpenAI/DeepSeek SDK 集成
    - 3.2.2 LLM 调用封装
    - 3.2.3 Token 计数和成本控制
  - 3.3 工具调用
    - 3.3.1 后端 Internal API 封装
    - 3.3.2 邮件查询工具
    - 3.3.3 标签应用工具
    - 3.3.4 优先级设置工具

- 3.4 核心功能实现
    - 3.4.1 邮件摘要生成
    - 3.4.2 回复草稿生成
    - 3.4.3 优先级预测
    - 3.4.4 标签建议
  - 3.5 可选功能
    - 3.5.1 垃圾邮件识别（规则版）
  - 3.6 Agent API 接口
    - 3.6.1 摘要生成接口
    - 3.6.2 草稿生成接口
    - 3.6.3 分析接口
- 4.前端开发（Vue 3）
  - 4.1 项目初始化
    - 4.1.1 Vite + Vue 3 工程搭建
    - 4.1.2 TypeScript 配置
    - 4.1.3 Element Plus 集成
    - 4.1.4 Pinia 状态管理
    - 4.1.5 Axios 配置
  - 4.2 认证与基础
    - 4.2.1 登录页面
    - 4.2.2 注册页面
    - 4.2.3 JWT 存储与刷新
    - 4.2.4 路由守卫
  - 4.3 主布局与导航
    - 4.3.1 侧边栏文件夹导航
    - 4.3.2 顶部工具栏
    - 4.3.3 用户菜单
    - 4.3.4 响应式设计
  - 4.4 邮件列表页面
    - 4.4.1 列表展示（收件箱/已发送/草稿等）
    - 4.4.2 分页与无限滚动
    - 4.4.3 已读状态展示
    - 4.4.4 星标与优先级展示
    - 4.4.5 快速操作按钮
  - 4.5 邮件详情页面
    - 4.5.1 邮件内容显示
    - 4.5.2 附件列表与下载
    - 4.5.3 邮件操作菜单
    - 4.5.4 已读状态同步
    - 4.5.5 回复按钮
  - 4.6 写信/回复功能
    - 4.6.1 写信对话框
    - 4.6.2 收件人/抄送/密送输入
    - 4.6.3 正文编辑器

- 4.6.4 附件上传
    - 4.6.5 发送按钮与加载态
    - 4.6.6 草稿自动保存
  - 4.7 附件功能
    - 4.7.1 附件上传组件
    - 4.7.2 上传进度显示
    - 4.7.3 附件列表展示
    - 4.7.4 附件下载
  - 4.8 搜索与筛选
    - 4.8.1 搜索框组件
    - 4.8.2 关键词搜索
    - 4.8.3 高级筛选面板
    - 4.8.4 搜索结果展示
  - 4.9 标签功能
    - 4.9.1 标签管理页面
    - 4.9.2 创建标签
    - 4.9.3 给邮件打标签
    - 4.9.4 标签筛选
    - 4.9.5 标签颜色管理
  - 4.10 AI 功能面板
    - 4.10.1 摘要面板
    - 4.10.2 草稿面板
    - 4.10.3 优先级建议显示
    - 4.10.4 标签建议显示
    - 4.10.5 AI 加载状态
  - 4.11 其他页面
    - 4.11.1 用户设置页
    - 4.11.2 AI 配置页（开关）
    - 4.11.3 帮助页面
- 5.测试与质量
- 5.1 后端单元测试
    - 5.1.1 Auth 模块测试
    - 5.1.2 Mail 模块测试
    - 5.1.3 Mailbox 模块测试
    - 5.1.4 其他模块测试
  - 5.2 前端单元测试（可选）
    - 5.2.1 组件测试
    - 5.2.2 工具函数测试
  - 5.3 集成测试
    - 5.3.1 前后端集成测试
    - 5.3.2 AI 集成测试
    - 5.3.3 端到端流程测试
  - 5.4 性能测试

- 5.4.1 邮件列表加载性能
- 5.4.2 搜索性能
- 5.4.3 数据库索引优化
- 5.5 测试文档
  - 5.5.1 测试计划
  - 5.5.2 测试用例
  - 5.5.3 测试报告
- 5.6 演示与验收
  - 5.6.1 演示数据准备
  - 5.6.2 演示脚本编写
  - 5.6.3 验收标准检查

## 6.文档与交付

- 6.1 需求文档
- 6.2 设计文档
  - 6.2.1 概要设计
  - 6.2.2 详细设计
  - 6.2.3 数据库设计
  - 6.2.4 API 设计
- 6.3 实现文档
  - 6.3.1 后端实现说明
  - 6.3.2 前端实现说明
  - 6.3.3 AI 实现说明
- 6.4 部署与运维
  - 6.4.1 部署指南
  - 6.4.2 配置说明
  - 6.4.3 启动脚本
- 6.5 用户文档
  - 6.5.1 使用指南
  - 6.5.2 FAQ
- 6.6 周报与总结
  - 6.6.1 每周周报
  - 6.6.2 项目总结
- 6.7 答辩材料
  - 6.7.1 答辩 PPT
  - 6.7.2 演示视频脚本
  - 6.7.3 亮点展示

下表详细描述了各主要任务及其分配：

| 任务编号 | 任务名称 | 任务描述 | 负责人 | 工作量 | 达到指标 | 备注 |
|------|------|------|-----|-----|------|----|
|      |      |      |     |     |      |    |

| 任务编号 | 任务名称   | 任务描述                                       | 负责人            | 工作量   | 达到指标                      | 备注                                 |
|------|--------|--------------------------------------------|----------------|-------|---------------------------|------------------------------------|
| 1.1  | 需求分析   | 整理课程需求，编写需求分析文档                            | 组长 / 架构师       | 2 天   | 完成需求文档 1 份                | 包含功能需求、非功能需求                       |
| 1.2  | 系统架构设计 | 梳理系统模块划分、前后端分离架构                           | 组长 / 架构师       | 1.5 天 | 完成架构图、模块说明                | 参考项目架构与技术选型.md                     |
| 1.3  | 数据库设计  | 设计 ER 图、数据表结构、建立数据字典                       | 后端 1           | 2 天   | 完成 ER 图、SQL 创建脚本          | 核心表：user、mail_message、mailbox_item |
| 1.4  | 接口设计   | 编写 OpenAPI/Swagger 规范，定义请求 / 响应格式          | 架构师 + 前后端      | 2 天   | 完成接口文档 (Markdown/Swagger) | 包含认证、邮件、附件、标签、AI 接口                |
| 1.5  | 项目初始化  | 后端 Spring Boot、前端 Vite、Agent FastAPI 工程初始化 | 后端 1 / 前端 1/AI | 1 天   | 三个服务都能启动                  | 确保依赖正确安装                           |

W1 阶段检查清单：

- [ ] 需求文档完成
- [ ] ER 图评审通过

- [ ] 接口文档完成并团队认可

- [ ] 三个服务工程可启动

---

| 任务编号 | 任务名称                  | 任务描述                                     | 负责人  | 工作量   | 达到指标                                    | 备注                                              |
|------|-----------------------|------------------------------------------|------|-------|-----------------------------------------|-------------------------------------------------|
| 2.1  | 数据库连接与初始化             | 配置 MySQL 数据源、创建所有表、设置索引                  | 后端 1 | 1 天   | 数据库能正常连接，表创建成功                          | 运行 DDL 脚本无错误                                    |
| 2.2  | 通用响应与异常处理             | 实现统一的 API 响应格式、全局异常处理器                   | 后端 1 | 0.5 天 | 完成 ApiResponse 、 GlobalExceptionHandler | 参考架构文档中的响应格式                                    |
| 2.3  | Spring Security + JWT | 集成 Spring Security、实现 JWT Token 生成验证刷新   | 后端 1 | 1.5 天 | 完成 TokenService 、 AuthTokenFilter       | 支持 Bearer Token 认证                              |
| 2.4  | 用户注册接口                | 实现 /api/v1/auth/register，支持邮箱、用户名、密码     | 后端 1 | 1 天   | 能注册新用户，邮箱唯一性校验                          | 密码 BCrypt 哈希                                    |
| 2.5  | 用户登录接口                | 实现 /api/v1/auth/login，返回 JWT             | 后端 1 | 1 天   | 能登录返回 Token，Token 能被验证                  | 包含密码验证                                          |
| 2.6  | 当前用户接口                | 实现 /api/v1/users/me，获取当前登录用户信息           | 后端 1 | 0.5 天 | 能获取用户基本信息                               | 需 JWT 认证                                        |
| 2.7  | 发送邮件接口                | 实现 /api/v1/emails POST，支持收件人、抄送、主题、正文、附件 | 后端 2 | 2 天   | 能保存邮件到数据库，为发件人和收件人生成邮箱条目                | 涉及 mail_message 、 mail_recipient 、 mailbox_item |

| 任务编号 | 任务名称   | 任务描述                                                   | 负责人  | 工作量   | 达到指标                   | 备注              |
|------|--------|--------------------------------------------------------|------|-------|------------------------|-----------------|
| 2.8  | 邮件详情接口 | 实现 <code>/api/v1/emails/{id}</code> GET, 返回邮件内容、附件、收件人 | 后端 2 | 1.5 天 | 能获取完整邮件信息              | 包含正文和附件元信息      |
| 2.9  | 邮箱列表接口 | 实现 <code>/api/v1/mailbox</code> GET, 支持分页、文件夹筛选、关键词搜索  | 后端 2 | 2 天   | 能按文件夹拉取邮件列表, 支持分页和基本搜索 | 使用 MyBatis-Plus |

W2 阶段检查清单:

- [ ] 数据库表创建完成
- [ ] Auth 模块测试通过 (注册、登录、JWT)
- [ ] 发送邮件能保存到数据库
- [ ] 邮箱列表能查询返回
- [ ] Postman 或接口文档能验证上述接口

---

| 任务编号 | 任务名称      | 任务描述                                                   | 负责人  | 工作量   | 达到指标                                    | 备注     |
|------|-----------|--------------------------------------------------------|------|-------|-----------------------------------------|--------|
| 3.1  | 标记已读 / 未读 | 实现 PATCH <code>/api/v1/mailbox/items/{id}/read</code>  | 后端 2 | 1 天   | 能更新 <code>mailbox_item.read_flag</code> | 支持批量操作 |
| 3.2  | 星标邮件      | 实现 PATCH <code>/api/v1/mailbox/items/{id}/star</code>  | 后端 2 | 0.5 天 | 能更新 <code>mailbox_item.star_flag</code> | -      |
| 3.3  | 删除邮件      | 实现 DELETE <code>/api/v1/mailbox/items/{id}</code> , 软删 | 后端   | 0.5 天 | 能移动到垃圾箱, 不直接删除                          | 参考架构文档 |

| 任务编号 | 任务名称       | 任务描述                                                 | 负责人  | 工作量   | 达到指标                      | 备注                             |
|------|------------|------------------------------------------------------|------|-------|---------------------------|--------------------------------|
|      |            | 除到 TRASH                                             | 2    |       |                           |                                |
| 3.4  | 附件上传       | 实现 POST /api/v1/attachments, 保存文件到磁盘, 元信息入库          | 后端 2 | 1.5 天 | 能上传附件, 文件存储在 storage/ 目录  | 使用 multipart/form-data         |
| 3.5  | 附件下载       | 实现 GET /api/v1/attachments/{id}, 返回文件字节流             | 后端 2 | 1 天   | 能下载附件, 返回正确的 Content-Type | -                              |
| 3.6  | 前端项目初始化    | 初始化 Vite + Vue 3 + TypeScript + Element Plus + Pinia | 前端 1 | 1.5 天 | 项目能启动, 基础依赖安装完成           | npm run dev 能运行                |
| 3.7  | 前端路由配置     | 配置 Vue Router, 创建基础路由结构                              | 前端 1 | 1 天   | 能路由切换, 页面框架搭建             | /login、/register、/mail/inbox 等 |
| 3.8  | Pinia 状态管理 | 创建 AuthStore、MailboxStore、ComposeStore               | 前端 1 | 1.5 天 | 能管理全局状态, 支持状态持久化          | Token 保存到 localStorage         |
| 3.9  | 主布局框架      | 设计 AppLayout, 包括侧边栏、顶部栏、主内容区                         | 前端 1 | 1.5 天 | 布局美观, 响应式设计, 能切换文件夹       | 使用 Element Plus 组件             |
| 3.10 | 登录 / 注册页   | 实现登录和注册表单, 调用后端 API                                  | 前端 1 | 1 天   | 能注册和登录, Token 正确保存        | 包含表单验证                         |

| 任务编号 | 任务名称     | 任务描述                              | 负责人  | 工作量   | 达到指标             | 备注          |
|------|----------|-----------------------------------|------|-------|------------------|-------------|
|      | 面        |                                   |      |       |                  |             |
| 3.11 | Axios 配置 | 配置 Axios 拦截器，自动添加 Authorization 头 | 前端 1 | 0.5 天 | 能自动携带 JWT 调用 API | 支持请求 / 响应拦截 |

---

| 任务编号 | 任务名称   | 任务描述                      | 负责人  | 工作量   | 达到指标                 | 备注                 |
|------|--------|---------------------------|------|-------|----------------------|--------------------|
| 4.1  | 邮件列表页面 | 实现邮件列表展示，支持分页、筛选、已读状态显示   | 前端 2 | 2 天   | 列表能显示邮件，支持点击查看详情     | 使用 Table 或 List 组件 |
| 4.2  | 邮件详情页面 | 显示邮件完整内容、附件、收件人信息         | 前端 2 | 1.5 天 | 能正确显示邮件正文和附件         | 支持附件下载             |
| 4.3  | 写信对话框  | 实现写信弹窗，支持收件人输入、正文编辑、附件上传  | 前端 2 | 2 天   | 能填写并发送邮件             | 包括草稿自动保存           |
| 4.4  | 搜索与筛选  | 实现搜索框、高级筛选面板（日期、发件人、文件夹等） | 前端 2 | 1.5 天 | 能搜索邮件，支持多条件筛选        | -                  |
| 4.5  | 标签管理   | 前端标签管理页面、打标签、标签筛选         | 前端 2 | 1 天   | 能创建标签、给邮件打标签         | 支持颜色管理             |
| 4.6  | 标签接口   | 后端实现标签 CRUD               | 后端   | 1.5   | 完成 label、label_rel 表 | -                  |

| 任务编号 | 任务名称            | 任务描述                               | 负责人       | 工作量   | 达到指标                          | 备注          |
|------|-----------------|------------------------------------|-----------|-------|-------------------------------|-------------|
|      | 开发              | 和邮件标签关系                            | 2         | 天     | 的接口                           |             |
| 4.7  | 搜索接口开发          | 后端实现搜索接口, 支持多条件组合                  | 后端 2      | 1.5 天 | /api/v1/emails/search 能返回搜索结果 | -           |
| 4.8  | Agent 框架搭建      | 初始化 FastAPI, 配置 LLM SDK, 设置路由      | AI        | 1.5 天 | Agent 服务能启动, 能调用 LLM API      | 配置好 .env    |
| 4.9  | Internal API 封装 | Agent 封装后端 Internal API 调用, 实现工具函数 | AI        | 1.5 天 | Agent 能调用后端接口获取邮件             | 包含认证和错误处理   |
| 4.10 | 邮件摘要功能          | Agent 实现邮件摘要生成, 后端实现摘要接口           | AI + 后端 2 | 1.5 天 | 能生成邮件摘要, 保存到数据库               | 使用 LLM 工具调用 |
| 4.11 | 回复草稿功能          | Agent 实现草稿生成, 后端实现草稿接口             | AI + 后端 2 | 1.5 天 | 能生成回复草稿                       | -           |
| 4.12 | 优先级与标签建议        | Agent 实现优先级预测和标签推荐                 | AI + 后端 2 | 1.5 天 | 能推荐优先级和标签                     | -           |
| 4.13 | AI 面板前端         | 前端显示摘要、草稿、优先级建议                    | 前端 2      | 1.5 天 | AI 结果能在邮件详情页显示                | 支持刷新加载      |
| 4.14 | 整体联调            | 前后端 + Agent 全流程联调测试                | 全体        | 1 天   | 端到端流程能正常工作                    | 包含错误处理      |

W4 阶段检查清单:

- [ ] 前端页面完整, 功能基本完成
- [ ] 前端能正常调用后端 API

- [ ] Agent 能成功启动和调用 LLM
- [ ] 邮件摘要、草稿、优先级建议能生成
- 

| 任务编号 | 任务名称   | 任务描述                         | 负责人      | 工作量   | 达到指标               | 备注                        |
|------|--------|------------------------------|----------|-------|--------------------|---------------------------|
| 5.1  | 后端单元测试 | 编写 Auth、Mail、Mailbox 模块的单元测试 | 后端 1/2   | 1.5 天 | 测试覆盖率 > 70%        | JUnit 5 + Mockito         |
| 5.2  | 前端单元测试 | 编写关键组件和工具函数测试                | 前端 1/2   | 1 天   | 测试覆盖率 > 60%        | Vitest + Vue Test Utils   |
| 5.3  | 集成测试   | 前后端集成测试、AI 集成测试              | 全体       | 2 天   | 主流程能完整运行           | 包含注册、发送、接收、AI 分析          |
| 5.4  | 性能测试   | 邮件列表加载性能、搜索性能、数据库索引优化        | 后端 1/2   | 1.5 天 | 列表加载 < 1s, 搜索 < 2s | 使用 ab 或 JMeter            |
| 5.5  | 缺陷修复   | 修复测试过程中发现的 bug               | 全体       | 1.5 天 | 所有 P0/P1 缺陷修复      | 记录在缺陷跟踪表                  |
| 5.6  | 安全审查   | SQL 注入防御、XSS 防护、认证授权检查       | 后端 1     | 1 天   | 通过安全审查             | 参考 OWASP Top 10           |
| 5.7  | 代码审查   | 代码质量审查、命名规范、注释完整性            | 组长 / 架构师 | 1 天   | 代码符合规范             | 使用 SonarQube 或 Checkstyle |
| 5.8  | 演示数据准备 | 创建演示账号、示例邮件、示例附件             | AI / 测试  | 0.5 天 | 演示数据完整可用           | 包含不同文件夹、标签、AI 结果          |
| 5.9  | 测试文档   | 编写测试计划、测试用例、测试报告             | AI / 测试  | 1.5 天 | 完成测试文档             | 包含测试覆盖范围、结果总结             |
| 5.10 | 部署文档   | 编写部署指南、启动脚本                  | 组长 /     | 1 天   | 完成部署文档             | 支持一键启动                    |

| 任务编号 | 任务名称      | 任务描述                         | 负责人    | 工作量   | 达到指标           | 备注         |
|------|-----------|------------------------------|--------|-------|----------------|------------|
|      |           | 本、配置说明                       | 后端 1   |       |                |            |
| 5.11 | API 文档最终化 | 补充 API 文档缺失部分, 生成 Swagger UI | 架构师    | 0.5 天 | API 文档完整清晰     | 支持在线测试     |
| 5.12 | 用户指南      | 编写简要使用说明和 FAQ                | 前端 1/2 | 0.5 天 | 用户能理解系统操作      | -          |
| 5.13 | 答辩 PPT    | 编写答辩演讲稿, 制作 PPT              | 组长     | 1 天   | PPT 逻辑清晰, 亮点突出 | 包含架构图、演示截图 |
| 5.14 | 演示与验收     | 最终演示、问题修复、验收签字               | 全体     | 1 天   | 通过验收           | -          |

W5 阶段检查清单:

- [ ] 单元测试完成并通过
- [ ] 集成测试通过, 主流程可用
- [ ] 所有 P0 缺陷已修复
- [ ] 文档完整, 部署脚本可用
- [ ] 演示数据准备完毕
- [ ] 答辩 PPT 完成

---

#### 4.3 任务分配与角色职责

基于 5-6 人小组分工:

| 岗位       | 姓名 | 主要职责                | 关键任务                            |
|----------|----|---------------------|---------------------------------|
| 组长 / 架构师 | -  | 需求拆解、架构设计、进度管理、最终集成 | 1.1-1.5, 5.6-5.8, 5.13-5.14     |
| 后端 1     | -  | 用户认证、权限、基础设施、通用模块   | 1.3, 2.1-2.3, 2.6, 5.1, 5.6-5.7 |

| 岗位      | 姓名 | 主要职责                   | 关键任务                                 |
|---------|----|------------------------|--------------------------------------|
| 后端 2    | -  | 邮件收发、邮箱管理、附件、搜索、标签     | 2.7-2.9, 3.1-3.5, 4.6-4.7, 4.10-4.12 |
| 前端 1    | -  | 工程初始化、路由、状态管理、主布局、基础页面 | 3.6-3.11, 4.4-4.5                    |
| 前端 2    | -  | 邮件详情、写信、附件、搜索、AI 面板    | 4.1-4.5, 4.13                        |
| AI / 测试 | -  | Agent 服务、AI 功能、测试、演示数据 | 4.8-4.12, 5.2-5.5, 5.8-5.9           |

---

## 5. 风险管理与应对

| 序号 | 风险项           | 风险描述                 | 发生概率 | 影响程度 | 应对方案                               |
|----|---------------|----------------------|------|------|------------------------------------|
| R1 | 前后端接口契约不一致    | 前端和后端对接口理解不同，导致集成困难  | 中    | 高    | 提前编写详细接口文档，定期接口评审，Mock 数据          |
| R2 | 数据库 ER 图设计有缺陷 | 邮件和用户邮箱视图分离设计不当，导致重构 | 低    | 高    | W1 邀请有数据库经验的人评审，制作关系图              |
| R3 | AI 调用成本过高     | OpenAI API 调用费用超预期   | 中    | 中    | 使用 DeepSeek 等便宜模型，设置 Token 限额，监控费用 |
| R4 | LLM API 服务不稳定 | 无法连接 LLM 服务，影响 AI 功能 | 中    | 中    | 实现降级方案，基础邮件不依赖 AI，设置超时重试           |
| R5 | 人员请假或离职       | 关键人员缺席，影响进度          | 低    | 中    | 跨团队代码评审，知识转移，建立文档                  |
| R6 | 附件存储路径        | 附件存储安全性、权限问          | 低    | 中    | 使用 UUID 隔离文件名，设置严                  |

| 序号  | 风险项            | 风险描述             | 发生概率 | 影响程度 | 应对方案                       |
|-----|----------------|------------------|------|------|----------------------------|
|     | 问题             | 题                |      |      | 格的目录权限                     |
| R7  | JWT Token 过期处理 | Token 刷新逻辑复杂或有漏洞 | 低    | 中    | 提前制定 Token 刷新策略，编写测试用例     |
| R8  | 集成测试延期         | 各模块完成进度不一致，难以集成  | 中    | 中    | 定期集成点（每周 1-2 次），提前 Mock 接口 |
| R9  | 前端性能问题         | 邮件列表加载慢，搜索结果延迟高  | 中    | 低    | 实现虚拟滚动、客户端缓存、后端索引优化        |
| R10 | 答辩时间不足         | 演示功能太多，答辩超时      | 低    | 低    | 提前准备演示脚本，精简演示内容，时间控制       |

## 5.2 进度监控

- 每周进度检查：周五统计本周完成的任务和遗留问题
- 关键节点验收：W1/W2/W3/W4 末进行阶段验收
- 风险预警：如有任务逾期 > 1 天，立即进行风险评估和调整
- 留白时间：W5 预留 2-3 天用于缓冲和应急

---

## 6. 资源与预算

| 资源       | 数量  | 说明       |
|----------|-----|----------|
| 组长 / 架构师 | 1 人 | 全职，5-6 周 |
| 后端开发     | 2 人 | 全职，5-6 周 |
| 前端开发     | 2 人 | 全职，5-6 周 |
| AI / 测试  | 1 人 | 全职，5-6 周 |
| 合计       | 6 人 | 5-6 周    |

## 6.2 技术资源

| 资源       | 预算       | 说明                   |
|----------|----------|----------------------|
| 开发主机     | 0        | 学校机房或个人电脑            |
| 服务器（演示用） | 0        | 学校服务器或免费云主机          |
| LLM API  | 可选       | DeepSeek 或 OpenAI 均可 |
| 数据库      | 0        | MySQL 开源免费           |
| 合计       | 约 0-100¥ | 可选 API 费用            |

---

## 7. 成功标准与验收条件

### 7.1 功能验收标准

必做功能（共 12 项）

1. 用户能正常注册、登录、登出
2. 用户能发送邮件给其他用户
3. 用户能查看收件箱邮件列表
4. 用户能查看已发送邮件列表
5. 用户能查看邮件详情（包括正文、收件人、附件）
6. 用户能上传附件（邮件时）
7. 用户能下载附件
8. 用户能标记邮件为已读/未读
9. 用户能删除邮件（软删除到垃圾箱）
10. 用户能搜索邮件（关键词、发件人等）
11. 用户能创建和使用标签
12. 邮件正文存数据库，附件存磁盘，元信息入库

展示型 AI 功能（共 3 项）

1. 用户能查看邮件摘要
2. 用户能查看回复草稿建议
3. 系统能推荐邮件优先级和标签

代码与文档标准

- 代码符合规范，注释完整
- 接口文档完整准确
- 数据库设计文档齐全
- 测试覆盖率 > 60%
- 部署文档能指导独立部署

### 7.2 性能指标

| 指标     | 目标      | 说明           |
|--------|---------|--------------|
| 邮件列表加载 | < 1s    | 加载 20 条邮件    |
| 邮件详情加载 | < 500ms | 包括正文和附件      |
| 搜索响应   | < 2s    | 1000 条邮件库中搜索 |
| 登录响应   | < 500ms | 包括 Token 生成  |
| 摘要生成   | < 5s    | 首次生成 AI 摘要   |

### 7.3 质量指标

| 指标        | 目标           |
|-----------|--------------|
| 代码审查通过率   | 100%         |
| Bug 密度    | < 3 个 / 千行代码 |
| 测试通过率     | 100%         |
| 文档完整率     | 100%         |
| 团队代码规范符合率 | > 95%        |

## 8. 团队协作与沟通

### 8.1 会议与汇报

| 会议 | 频率         | 参与者 | 内容             |
|----|------------|-----|----------------|
| 晨会 | 每日 10 分钟   | 全体  | 进度更新、今日计划、阻碍反馈 |
| 周会 | 每周 1 次（周五） | 全体  | 周总结、下周计划、问题讨论  |

| 会议    | 频率       | 参与者  | 内容             |
|-------|----------|------|----------------|
| 技术评审会 | 每周 1-2 次 | 相关模块 | 代码审查、架构讨论、问题解决 |
| 集成测试会 | W3/W4 末  | 全体   | 集成测试结果、缺陷讨论    |

## 8.2 沟通工具与文档

- 源代码：Git（GitHub/GitLab），分支管理规范
- 文档共享：项目 Wiki、Markdown 文件在 docs/ 目录
- 任务追踪：Issue 或项目管理工具（Jira/Linear）
- 即时沟通：微信/钉钉 群组
- 代码审查：Pull Request + 代码评论

## 8.3 代码分支规范

```

```bash
main                # 主分支，发布版本
├─ develop          # 开发分支，集成测试基础
│   ├─ feature/auth  # 认证功能分支
│   ├─ feature/mail  # 邮件功能分支
│   ├─ feature/frontend # 前端功能分支
│   ├─ feature/ai     # AI 功能分支
│   └─ bugfix/xxx     # 缺陷修复分支
└─ release/v1.0      # 发布分支
命名规范：feature/{模块}-{功能}，如 feature/auth-jwt-token
提交规范：[模块] 功能说明，如 [auth] 实现 JWT Token 刷新
```

```

---

## 9. 附录

### 9.1 关键日期

| 日期     | 事项                   |
|--------|----------------------|
| 第 1 周末 | W1 阶段验收（架构、数据库、接口文档） |
| 第 2 周末 | W2 阶段验收（邮件收发闭环）      |
| 第 3 周末 | W3 阶段验收（前端基础框架完成）    |
| 第 4 周末 | W4 阶段验收（AI 集成、全功能联调） |

| 日期     | 事项   |
|--------|------|
| 第 5 周末 | 最终答辩 |

## 9.2 常用命令速查

后端

```
```bash
```

```
cd backend
```

```
mvn clean install
```

```
mvn spring-boot:run
```

```
```
```

前端

```
```bash
```

```
cd frontend
```

```
npm install
```

```
npm run dev
```

```
```
```

Agent

```
```bash
```

```
cd agent
```

```
pip install -r requirements.txt
```

```
python -m uvicorn app.main:app --reload
```

```
```
```

## 9.3 参考文档

- 项目架构与技术选型: [项目架构与技术选型.md](项目架构与技术选型.md)

- CLAUDE.md 协作说明: [CLAUDE.md](../CLAUDE.md)

- 实训课程要求: [doc/2026 年实训要求.pdf](../doc/2026 年实训要求.pdf)

## 9.4 常见问题 FAQ

Q: 如果某个模块逾期怎么办?

A: 立即评估对下游模块的影响, 启动应急方案 (如 Mock 接口、或调整优先级)。

Q: 前后端接口不一致如何处理?

A: 停止开发, 召集前后端开会对齐, 修改接口文档并重新评审。

Q: LLM API 费用太高怎么办?

A: 切换到更便宜的模型 (DeepSeek), 或降低调用频率, 或在测试环境使用本地模型。

Q: 时间不足, 如何取舍?

A: 优先保证必做功能, 可舍弃加分项或简化 AI 功能 (如先做规则版垃圾邮件识别)。

---

文档版本: 1.0

编制人: 李明轩

审核人: 张杨

最后修改时间: 2026-06-06